Q1 クエリパラメータによる数値フィルタリング

`index.php` に以下を実装せよ。

GETパラメータ **`min`** と **`max`** を受け取り、**`min`** 以上 **`max`** 以下の整数を **`<ul>`** リストとして出力する。**`min`** / **`max`** が未指定・数値以外の場合は「パラメータが不正です」を表示する。

```
http://localhost/index.php?min=3&max=7
```

```
<ul>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
</ul>
```

Q2 CSV読み込みとHTML表示

`data.csv` を読み込み、HTMLテーブルとして出力する **`table.php`** を作成せよ。1行目はヘッダ行として **`<thead>`** に、2行目以降は **`<tbody>`** に出力すること。XSS対策も施すこと。

```
id,name,score
1,田中,85
2,鈴木,92
3,佐藤,78
```

Q3 フォームバリデーションと再表示

`form.php` を1ファイルで実装せよ（送信先も同ファイル）。

フィールド名	バリデーション
`username`	必須・1〜20文字
`email`	必須・メール形式
`age`	必須・1〜120の整数

GETアクセス時は空フォームを表示する。POSTで送信後、エラーがある場合は入力値を保持したまま各フィールドの下にエラーメッセージを赤字で表示する。全項目が正常な場合は「送信完了：{username}さん（{email}）、年齢{age}歳」と表示する。

Q4 RGB → HEX 変換 API

GETパラメータ `r`・`g`・`b`（各 0〜255 の整数）を受け取り、JSONを返す `color.php` を作成せよ。レスポンスヘッダに `Content-Type: application/json` を設定し、HEXは大文字で出力すること。

```
// 正常時
{ "status": "ok", "hex": "#FF6347", "rgb": { "r": 255, "g": 99, "b": 71 } }

// エラー時（範囲外・未指定）
{ "status": "error", "message": "Invalid parameter" }
```

Q5 セッションを使ったログイン機能

`login.php` と `mypage.php` を作成せよ。

`login.php` はフォームにIDとパスワードを入力して送信する。ID: `admin` / パスワード: `pass1234` の場合のみログイン成功とし、成功時はセッションにユーザー情報を保存して `mypage.php` へリダイレクトする。失敗時は「IDまたはパスワードが違います」を表示しフォームを再表示する。

`mypage.php` はセッションが存在する場合に「ようこそ、adminさん」とログアウトボタンを表示する。セッションが存在しない場合は `login.php` へリダイレクトする。ログアウトボタン押下でセッション破棄後 `login.php` へリダイレクトする。

Q6 JSONファイルへの掲示板書き込み・読み込み

`bbs.php` を1ファイルで実装せよ。

投稿フォーム（名前・本文）からPOSTで受信し **`bbs.json`** に追記保存する。投稿一覧は新着順（id降順）でHTMLに表示する。名前・本文どちらかが空の場合は「入力してください」とエラーを表示する。XSS対策も施すこと。

```
[
  { "id": 1, "name": "田中", "body": "こんにちは", "created_at": "2025-10-17 14:30:00" },
  { "id": 2, "name": "鈴木", "body": "はじめまして", "created_at": "2025-10-17 15:00:00" }
]
```

Q7 九九テーブル出力（パラメータ拡張版）

`kuku.php` に以下を実装せよ。

GETパラメータ **`n`** で段数の上限を受け取り（デフォルト9、最大20）、**`n × n`** の掛け算テーブルをHTMLテーブルで出力する。ヘッダ行・ヘッダ列は **`<th>`** で出力し、積は **`<td>`** に出力する。**`n`** が1未満・20超・数値以外の場合は **`n=9`** にフォールバックする。積が100以上のセルにはCSSクラス **`highlight`**（文字色赤）を付与する。

Q8 MySQL DBからのデータ取得とフィルタリング

以下のデータベースに接続し、**`items`** テーブルの内容を取得してHTMLテーブルで表示すること。

項目	値
Host	**`localhost`**
User	**`web`**

項目	値
Password	**`compe`**
Database	**`speedtest`**
Table	**`items`**

```
CREATE TABLE items (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  price INT NOT NULL
);
```

GETパラメータ **`category`** が指定されている場合は、そのカテゴリに一致するレコードのみを表示すること。指定がない場合は全件を表示すること。表示順は **`price`** の昇順とする。

出力項目は **`id`**、**`name`**、**`category`**、**`price`** とし、**`price`** は **`1000円`** のように **`円`** を付けて表示すること。

SQLインジェクション対策を施すこと。DB接続または取得に失敗した場合は「データの取得に失敗しました」と表示すること。

Q9 画像アップロードと拡張子・サイズバリデーション

`upload.php` を実装せよ。

許可拡張子は **`jpg`**・**`jpeg`**・**`png`**・**`gif`**、最大ファイルサイズは2MBとする。バリデーション通過後は **`uploads/`** ディレクトリに **`{タイムスタンプ}_{元のファイル名}`** の形式で保存し「アップロード成功：{ファイル名}」を表示する。

エラー条件	メッセージ
拡張子不正	「許可されていないファイル形式です」
サイズ超過	「ファイルサイズが2MBを超えています」

エラー条件	メッセージ
アップロード失敗	「アップロードに失敗しました」

Q10 REST風 JSON API（CRUD の一部）

`api.php` に以下のメモAPIを実装せよ。データ永続化は **`notes.json`** ファイルで行う。

メソッド	パス	処理
`GET`	**`/api.php`**	全メモ一覧を返す
`GET`	**`/api.php?id={id}`**	指定IDのメモを返す
`POST`	**`/api.php`**	メモ新規追加（bodyに **`title`**・**`content`** をJSON送信）
`DELETE`	**`/api.php?id={id}`**	指定IDのメモ削除

リクエストボディのJSONは **`file_get_contents('php://input')`** で取得する。**`Content-Type: application/json`** を必ずセットする。存在しないIDは **`404 Not Found`**、未対応メソッドは **`405 Method Not Allowed`**、**`title`** / **`content`** が空の場合は **`400 Bad Request`** を返す。

```
// 正常時の共通フォーマット
{ "status": "ok", "data": { ... } }

// エラー時
{ "status": "error", "message": "Not found" }
```

Powered by **https://markdowntoimage.com**